

Task 1: Git Merge

- **What is a fast-forward merge?**
Moving the branch pointer forward to the latest commit because the history has not diverged.
 - **When does Git create a merge commit instead?**
When both branches have new, unique commits since they split apart, requiring a new commit to tie them together.
 - **What is a merge conflict?**
A collision that happens when you modify the exact same line of the same file differently on two separate branches.
-

Task 2: Git Rebase

? What does rebase actually do to your commits?

It lifts your branch commits, pauses them, catches your branch up to the latest main commit, and then reapplies your commits one by one on top.

? How is the history different from a merge?

Merge preserves the exact historical timeline and visual branch structures. Rebase rewrites history into a clean, single file line.

? Why should you never rebase commits that have been pushed and shared with others?

It changes the unique ID hashes of existing commits, which breaks the repository history for team members who pulled the old commits.

? When would you use rebase vs merge?

Use **Rebase** locally to clean up your personal branch before sharing. Use **Merge** on public/shared branches to keep an accurate historical record.

Task 3: Squash Commit vs Merge Commit

- **What does squash merging do?**
It condenses a long string of work commits into one single commit before merging it into the target branch.
 - **When would you use squash merge vs regular merge?**
Use **Squash** for feature branches filled with messy typo fixes or small iterations. Use **Regular Merge** when every individual commit contains crucial, isolated historical value.
 - **What is the trade-off of squashing?**
You lose the granular step-by-step history and context of how the code evolved over time during development.
-

Task 4: Git Stash

- **What is the difference between git stash pop and git stash apply?**
pop restores your changes and deletes them from the stash list. apply restores your changes but keeps the backup saved in the stash list.
 - **When would you use stash in a real-world workflow?**
When you are deep in a complex feature, and your boss suddenly requests an urgent production hotfix that requires a clean branch switch.
-

Task 5: Cherry Picking

- **What does cherry-pick do?**
It copies a specific commit from any branch and applies it as a brand-new commit onto your currently active branch.
 - **When would you use cherry-pick in a real project?**
When a feature branch is abandoned or incomplete, but it contains one specific bug fix or helper function that you need right away on main.
 - **What can go wrong with cherry-picking?**
It generates duplicate commits with different hashes. It can also cause messy conflicts if the code depends on changes from skipped commits.
-